Course Recommendation API - Complete Integration Guide

Quick Start

API Base URL: http://127.0.0.1:8000 (local) or your deployed URL
Interactive Documentation: http://127.0.0.1:8000/docs
Health Check: http://127.0.0.1:8000/health

Test Your First Request

```
curl -X POST "http://127.0.0.1:8000/recommend?top_k=3" \
 -H "Content-Type: application/json" \
 -d '{
  "gender": "M",
  "stream": "Science",
  "mathematics": 85,
  "physics": 78,
  "interest_mathematics": 9,
  "analytical": 8
 }'
```

What This API Does

The Course Recommendation API analyzes student academic performance, interests, and aptitudes to recommend suitable undergraduate courses. It supports three academic streams: Science, Commerce, and Arts.

Key Features:

- Personalized course recommendations based on student data
- Stream-specific filtering and analysis
- Batch processing for multiple students
- Built-in validation and error handling
- Interactive API documentation

API Endpoints Reference

1. Health Check

GET /health

Response:

{

"status": "healthy",

"model_loaded": true,

"timestamp": "2024-01-15T10:30:00"

}

2. Single Student Recommendation

POST /recommend?top_k=5&filter_by_stream=true

Query Parameters:

- top_k (optional): Number of recommendations (1-20, default: 5)
- filter_by_stream (optional): Filter by student's stream (default: true)

Request Body:

{

"gender": "M",

"stream": "Science",

"physics": 85,

"chemistry": 78,

"mathematics": 92,

"biology": 70,

"computer_science": 80,

"english": 75,

"interest_physics": 8,

"interest_chemistry": 6,

"interest_mathematics": 9,

"interest_biology": 5,

"interest_computer_science": 7,

"interest_english": 6,

"analytical": 8,

```
 "verbal": 6,
 "creativity": 7,
 "numerical": 9
}
```


Response:


```
{
 "success": true,
 "student_info": {
  "gender": "M",
  "stream": "Science",
  "filter_by_stream": true,
  "key_subjects": [
   {"subject": "mathematics", "score": 92},
   {"subject": "physics", "score": 85}
  ],
  "top_interests": [
   {"subject": "mathematics", "interest": 9},
   {"subject": "physics", "interest": 8}
  ],
  "aptitude_strengths": [
   {"aptitude": "numerical", "score": 9},
   {"aptitude": "analytical", "score": 8}
  ]
 },
 "recommendations": [
  {"course": "B.Sc. Mathematics", "score": 0.845},
  {"course": "B.Sc. Physics", "score": 0.782},
  {"course": "B.Sc. Computer Science", "score": 0.654}
```

```
 ],
 "total_courses": 3
}
```


3. Batch Recommendations

POST /recommend/batch?top_k=3

Request Body:


```
[
 {
  "gender": "M",
  "stream": "Science",
  "mathematics": 90,
  "physics": 85
 },
 {
  "gender": "F",
  "stream": "Commerce",
  "economics": 88,
  "banking": 85
 }
]
```


Response:


```
{
 "success": true,
 "total_students": 2,
 "results": [
  {
   "student_index": 0,
   "gender": "M",
```

```
    "stream": "Science",
    "recommendations": [
     {"course": "B.Sc. Mathematics", "score": 0.823},
     {"course": "B.Sc. Physics", "score": 0.756}
    ]
   },
   {
    "student_index": 1,
    "gender": "F",
    "stream": "Commerce",
    "recommendations": [
     {"course": "B.Com. Economics", "score": 0.892},
     {"course": "B.Com. Banking and Insurance", "score": 0.734}
    ]
   }
 ]
}
```


4. Available Courses

GET /courses

Response:


```
{
  "success": true,
  "total_courses": 17,
  "by_stream": {
   "Science": [
    "B.Sc. Mathematics",
    "B.Sc. Physics",
    "B.Sc. Chemistry",
```

```
    "B.Sc. Biology",
    "B.Sc. Computer Science",
    "B.Sc. Statistics",
    "B.Sc. Botany",
    "B.Sc. Zoology"
  ],
  "Commerce": [
    "B.Com. General",
    "B.Com. Economics",
    "B.Com. Banking and Insurance",
    "B.Com. Accounting and Finance"
  ],
  "Arts": [
    "B.A. English",
    "B.A. History",
    "B.A. Political Science",
    "B.A. Sociology",
    "B.A. Education",
    "B.A. Geography"
  ]
 }
}
```


5. Model Information

GET /model/info

Response:


```
{
 "success": true,
 "model_type": "LightGBM Ranker",
```

```
 "features_count": 55,
 "student_features_count": 36,
 "use_subject_penalties": true,
 "penalty_strength": 0.6,
 "bonus_strength": 0.8,
 "top_features": [
  {"feature": "subject_course_alignment", "importance": 1245.2},
  {"feature": "science_perf_interest_corr", "importance": 892.1}
 ]
}
```


Student Data Format

Required Fields

- **gender**: "M" or "F"
- **stream**: "Science", "Commerce", or "Arts"

Subject Scores (0-100)

Only include subjects relevant to the student's stream:

Science Stream:

- physics, chemistry, mathematics, biology, computer_science, english

Commerce Stream:

- english, accountancy, business_studies, economics, banking

Arts Stream:

- english, history, political_science, sociology, education, geography, hindi

Interest Levels (0-10, Optional)

Rate student's interest in each subject:

- interest_physics, interest_chemistry, interest_mathematics, etc.
- 0 = No interest, 10 = Very high interest

Aptitude Scores (0-10, Optional)

- analytical: Problem-solving and logical reasoning

- verbal: Language and communication skills
- creativity: Creative and artistic thinking
- numerical: Mathematical and quantitative skills

Data Validation Rules

- All scores must be integers
- Subject scores: 0-100 range
- Interest/aptitude scores: 0-10 range
- Missing fields default to 0
- Invalid gender/stream values will return validation errors

Integration Examples

Python with requests

```
import requests
import json


def get_course_recommendations(student_data, api_url="http://127.0.0.1:8000"):
    try:
        response = requests.post(
            f"{api_url}/recommend",
            json=student_data,
            params={"top_k": 5}
        )
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
        print(f"Error: {e}")
        return None


# Example usage
```

```
student = {
  "gender": "F",
  "stream": "Commerce",
  "economics": 94,
  "banking": 88,
  "accountancy": 78,
  "interest_economics": 10,
  "interest_banking": 9,
  "analytical": 8,
  "numerical": 9
}

result = get_course_recommendations(student)
if result and result['success']:
  for rec in result['recommendations']:
    print(f"{rec['course']}: {rec['score']:.3f}")
```

JavaScript/TypeScript

```
async function getCourseRecommendations(studentData, apiUrl =
"http://127.0.0.1:8000") {
  try {
    const response = await fetch(` ${apiUrl}/recommend?top_k=5` , {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(studentData)
    });
```

```
        if (!response.ok) {
            throw new Error(` HTTP error! status: ${response.status}` );
        }

        const data = await response.json();
        return data;
    } catch (error) {
        console.error('Error:', error);
        return null;
    }
}

// Example usage
const student = {
    gender: "M",
    stream: "Science",
    mathematics: 92,
    physics: 85,
    chemistry: 78,
    interest_mathematics: 9,
    interest_physics: 8,
    analytical: 8,
    numerical: 9
};

getCourseRecommendations(student)
    .then(result => {
        if (result && result.success) {
```

```
        result.recommendations.forEach(rec => {
            console.log(` ${rec.course}: ${rec.score.toFixed(3)}` );
        });
    }
});
```

PHP with cURL

```
<?php
function getCourseRecommendations($studentData, $apiUrl = "http://127.0.0.1:8000")
{
    $url = $apiUrl . "/recommend?top_k=5";

    $ch = curl_init();
    curl_setopt($ch, CURLOPT_URL, $url);
    curl_setopt($ch, CURLOPT_POST, true);
    curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($studentData));
    curl_setopt($ch, CURLOPT_HTTPHEADER, [
        'Content-Type: application/json'
    ]);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

    $response = curl_exec($ch);
    $httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);

    if ($httpCode === 200) {
        return json_decode($response, true);
    }
```

```
    return null;
}

// Example usage
$student = [
    "gender" => "F",
    "stream" => "Arts",
    "english" => 85,
    "history" => 78,
    "political_science" => 82,
    "interest_english" => 8,
    "interest_history" => 7,
    "verbal" => 9,
    "creativity" => 8
];

$result = getCourseRecommendations($student);
if ($result && $result['success']) {
    foreach ($result['recommendations'] as $rec) {
        echo $rec['course'] . ": " . number_format($rec['score'], 3) . "\n";
    }
}
?>
```

Java with OkHttp

```
import okhttp3.*;
import com.google.gson.Gson;
import java.io.IOException;
```

```
public class CourseRecommendationClient {
    private static final String API_URL = "http://127.0.0.1:8000";
    private static final OkHttpClient client = new OkHttpClient();
    private static final Gson gson = new Gson();

    public static class StudentData {
        public String gender;
        public String stream;
        public Integer mathematics;
        public Integer physics;
        public Integer chemistry;
        public Integer interest_mathematics;
        public Integer interest_physics;
        public Integer analytical;
        public Integer numerical;
    }

    public static class RecommendationResponse {
        public boolean success;
        public Recommendation[] recommendations;

        public static class Recommendation {
            public String course;
            public double score;
        }
    }
```

```
  public static RecommendationResponse getRecommendations(StudentData
student) throws IOException {
    String json = gson.toJson(student);

    RequestBody body = RequestBody.create(
      MediaType.parse("application/json"), json
    );

    Request request = new Request.Builder()
      .url(API_URL + "/recommend?top_k=5")
      .post(body)
      .build();

    try (Response response = client.newCall(request).execute()) {
      if (response.isSuccessful()) {
        String responseBody = response.body().string();
        return gson.fromJson(responseBody, RecommendationResponse.class);
      }
      throw new IOException("API request failed: " + response.code());
    }
  }

  // Example usage
  public static void main(String[] args) {
    try {
      StudentData student = new StudentData();
      student.gender = "M";
      student.stream = "Science";
```

```
        student.mathematics = 92;
        student.physics = 85;
        student.chemistry = 78;
        student.interest_mathematics = 9;
        student.interest_physics = 8;
        student.analytical = 8;
        student.numerical = 9;

        RecommendationResponse result = getRecommendations(student);
        if (result.success) {
            for (RecommendationResponse.Recommendation rec :
result.recommendations) {
                System.out.printf("%s: %.3f%n", rec.course, rec.score);
            }
        }
    } catch (IOException e) {
        System.err.println("Error: " + e.getMessage());
    }
  }
}
```

Error Handling

HTTP Status Codes

- 200: Success
- 400: Bad Request (invalid data)
- 422: Validation Error (field constraints violated)
- 500: Internal Server Error

Error Response Format

```
{
```

```
 "success": false,
 "error": "Validation error",
 "detail": "Gender must be M or F"
}
```

Common Validation Errors

- "Gender must be M or F" - Invalid gender value
- "Stream must be Science, Commerce, or Arts" - Invalid stream
- "Subject scores must be between 0 and 100" - Score out of range
- "Interest levels must be between 0 and 10" - Interest out of range

Error Handling Example

```
def handle_api_response(response):
    try:
        data = response.json()
        if not data.get('success', False):
            error_msg = data.get('error', 'Unknown error')
            detail = data.get('detail', '')
            print(f"API Error: {error_msg}")
            if detail:
                print(f"Detail: {detail}")
            return None
        return data
    except ValueError:
        print("Invalid JSON response")
        return None
```

Sample Student Profiles

Science Student (Strong in Math & Physics)

```
{
 "gender": "M",
```

```
 "stream": "Science",
 "physics": 92,
 "chemistry": 78,
 "mathematics": 95,
 "biology": 65,
 "computer_science": 82,
 "english": 75,
 "interest_physics": 9,
 "interest_mathematics": 10,
 "interest_computer_science": 7,
 "analytical": 9,
 "numerical": 10,
 "verbal": 6,
 "creativity": 7
}
```


Commerce Student (Economics Focus)


```
{
 "gender": "F",
 "stream": "Commerce",
 "english": 82,
 "accountancy": 78,
 "business_studies": 85,
 "economics": 94,
 "banking": 91,
 "interest_economics": 10,
 "interest_banking": 9,
 "interest_business_studies": 8,
 "analytical": 8,
```

```
 "numerical": 9,
 "verbal": 8,
 "creativity": 6
}
```

Arts Student (Language & Social Sciences)

```
{
 "gender": "F",
 "stream": "Arts",
 "english": 88,
 "history": 82,
 "political_science": 85,
 "sociology": 78,
 "geography": 75,
 "hindi": 70,
 "interest_english": 9,
 "interest_history": 8,
 "interest_political_science": 9,
 "analytical": 7,
 "verbal": 10,
 "creativity": 9,
 "numerical": 5
}
```

Best Practices for Integration

1. Input Validation

Always validate data before sending to the API:

```
def validate_student_data(student):
    required_fields = ['gender', 'stream']
    for field in required_fields:
```

```
        if field not in student:
            return False, f"Missing required field: {field}"

    if student['gender'] not in ['M', 'F']:
        return False, "Gender must be M or F"

    if student['stream'] not in ['Science', 'Commerce', 'Arts']:
        return False, "Invalid stream"

    return True, "Valid"
```

2. Error Handling

Implement robust error handling:

```
async function safeApiCall(studentData) {
    try {
        const response = await fetch('/recommend', {
            method: 'POST',
            body: JSON.stringify(studentData),
            headers: {'Content-Type': 'application/json'}
        });

        if (!response.ok) {
            const errorData = await response.json();
            throw new Error(errorData.error || `HTTP ${response.status}`);
        }

        return await response.json();
    } catch (error) {
        console.error('API Error:', error.message);
```

```
    // Handle error appropriately for your application
    return { success: false, error: error.message };
  }
}
```

3. Performance Optimization

- Use batch endpoints for multiple students
- Cache results when appropriate
- Implement request timeouts
- Consider connection pooling for high-volume usage

4. Data Handling

- Only send non-zero scores to reduce payload size
- Validate score ranges before API calls
- Handle missing/optional fields gracefully

Testing Your Integration

1. Health Check First

```
curl http://127.0.0.1:8000/health
```

2. Test with Sample Data

Use the provided sample student profiles to verify your integration works correctly.

3. Test Error Cases

```
# Test invalid gender
curl -X POST "http://127.0.0.1:8000/recommend" \
  -H "Content-Type: application/json" \
  -d '{"gender": "X", "stream": "Science"}'

# Test invalid stream
curl -X POST "http://127.0.0.1:8000/recommend" \
  -H "Content-Type: application/json" \
  -d '{"gender": "M", "stream": "Invalid"}'
```

4. Performance Testing

Test with realistic data volumes to ensure your application can handle the expected load.

Deployment Considerations

For Production Use

- Replace 127.0.0.1:8000 with your actual API URL
- Implement proper authentication if required
- Set up monitoring and alerting
- Consider rate limiting on your end
- Use HTTPS in production environments

API Reliability

- The API includes built-in error handling
- Model predictions are cached for performance
- Health checks are available for monitoring
- Comprehensive logging for debugging

Support

Getting Help

1. Check the interactive documentation at /docs
2. Verify API health at /health
3. Review error messages in API responses
4. Test with provided sample data first

Common Integration Issues

- **Connection refused**: Ensure API server is running
- **Validation errors**: Check data types and ranges
- **Timeout errors**: Implement retry logic with backoff
- **JSON errors**: Verify Content-Type headers

Contact Information

For technical support or integration questions:

- Email: [your-email@domain.com]
- Documentation: Available at API /docs endpoint
- Health Status: Monitor via /health endpoint

This API provides a complete abstraction layer over the machine learning model, allowing you to integrate course recommendations into any application without needing to understand the underlying ML implementation.